

MUTAGENT

Turn your coding agent into an agentic AI engineer

Mutagent brings industry best practices to the entire agent development lifecycle. Interactively design your agent, pick the best framework for your case and prevent rework. Develop your evaluation suite from failures and calibrate it against domain experts. Diagnose production failures automatically and continuously improve your agents.

① Spec

② Build

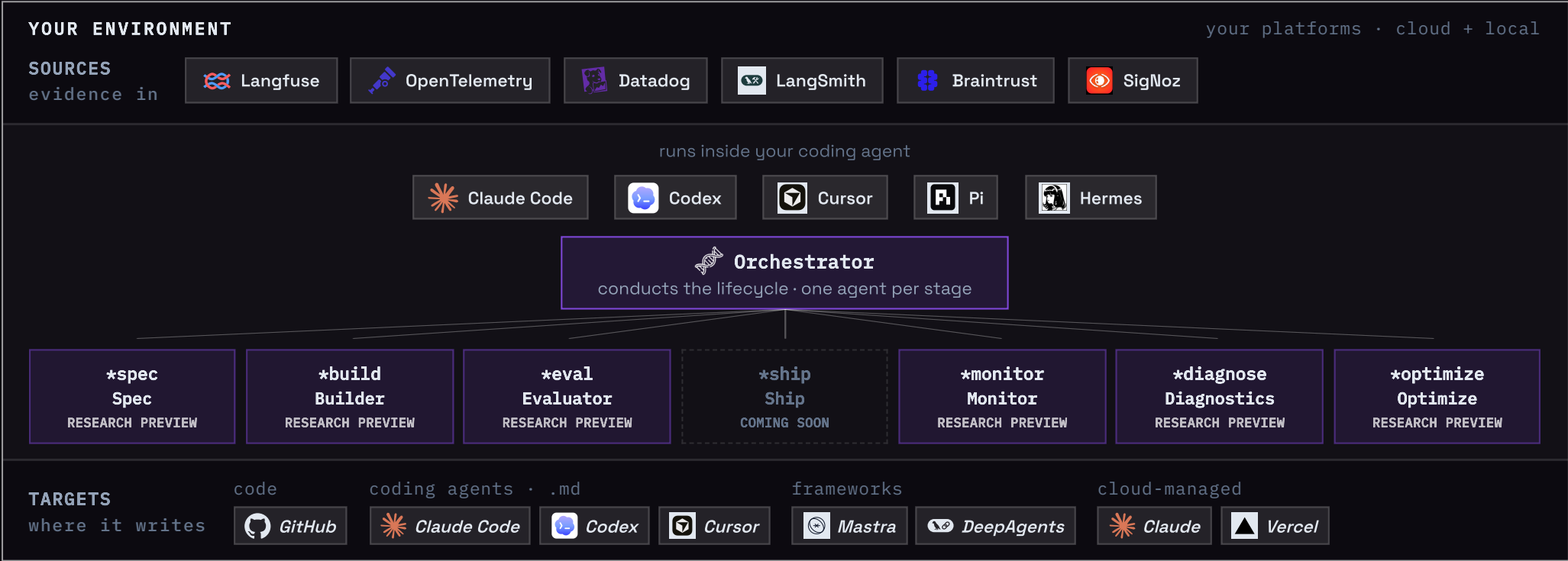
③ Evaluate

④ Diagnose

⑤ Optimize

System overview: the full agent development lifecycle

Mutagent runs model, framework, and observability-tool agnostic, locally inside your coding agent. Your traces and code never leave your machine.



Quick start

1 Open your project

A new folder or an existing repo. The system installs into it.

```
$ mkdir my-agents && cd my-agents  
# or: cd your-existing-repo
```

2 Install the system

Install the CLI, sign in, then add the agents and skills into .claude/ and .codex/.

```
$ npm install -g @mutagent/cli # or pnpm / bun  
$ mutagent login  
$ mutagent install helix
```

3 Boot

Start your coding agent, then summon orchestrator.

```
$ claude # or codex  
> *mutagent # or /mutagent-helix  
dashboard · lifecycle
```

What you'll see: the booted orchestrator

```
> *mutagent
```

```
M U T A G E N T · ADL orchestrator · Helix routes to your subagents
```

```
LIFECYCLE ① SPEC ② BUILD ③ EVALUATE ④ DIAGNOSE ⑤ OPTIMIZE  
SYSTEM    agentspec · builder · evaluator · diagnostics  
SETUP     not onboarded yet · run *onboard
```

```
COMMANDS  *spec *build *evaluate *diagnose *optimize *onboard *status  
Type a *command, or just say what you want.
```

How it works: one orchestrator, many subagents

Helix guides you through your agent development process. Jump in anywhere; nothing moves on without you.



Drive the lifecycle: say a stage, or just what you want

① SPEC	*spec	describe your agent, get a validated spec file
② BUILD	*build	turn the spec into a working agent
③ EVALUATE	*evaluate	check how good it is: pass or fail per behavior
④ DIAGNOSE	*diagnose	find out why it failed: ranked fixes
⑤ OPTIMIZE	gated	apply the fix, then re-check until it passes

First time? Start with ***onboard**

Add your provider keys, pick your workspace, and choose your models. Then jump into any stage.

Two ways to drive it: type a ***command**, or just say what you want. Helix routes it to the right stage and waits for you.

① Spec: turn an idea into a working spec

› ***spec** "design a Support Triage agent that reads Zendesk threads and drafts replies"

A short guided interview captures **what** your agent is, then writes a spec file you can build from. You answer with quick pick-lists.



```
› *spec
  What agent do you want to build?
› A Mastra agent that triages our support
  inbox and drafts replies for approval.
  (pick-lists) what kind? · how should it
  talk to your tools?
› (taps options)
  wrote your spec done · next: *build
  (I won't move on without you)
```

Features

Decide the right architecture first, in a 30-minute conversation, not after weeks of rework.

Explicit input and output contracts, so multi-agent handoffs can't fail silently.

A written, framework-agnostic spec the whole team can build from, not tribal knowledge.

📦 PRODUCES `your agent spec · agentspec.yaml`

AGENT
ai-architect
system design

② Build: your spec becomes a real, working agent

> ***build** "build the Support Triage agent from its spec on Mastra"



What the reviewer checks

A second pass confirms the build is faithful to your spec. Every capability you asked for is actually there, before you move on. No silent gaps.

Features

Generates against the spec, tests to green, with a reviewer that checks nothing is missing.

Every capability in the spec ships implemented and tested, never stubbed or silently skipped.

Model and dependency drift becomes a test failure, not a silent prod incident.



PRODUCES

a working agent, a baseline eval suite,
and tracing if you wire it

AGENT
ai-engineer
implementation

③ Evaluate: four ways to know if your agent is good

> ***evaluate** "evaluate the Support Triage agent on last week's 150 Zendesk threads"

Four ways to evaluate

FIND Suggests what's worth testing, mined from your runs.

SCORE Pass or fail per behavior. Built-in judge, or a test-suite in your own CI.

CREATE Write the checks yourself, guided by your spec.

DATA Turn a few examples into a realistic set of test cases.

Features

Evals mined from your real traces, plus synthetic cases generated from your spec when you need coverage you do not have yet.

A judge you can trust: same input, same verdict every run.

Your first eval set on day one, no blank-page cold start.

A stable pass/fail gate for CI, plus a continuous score for direction.

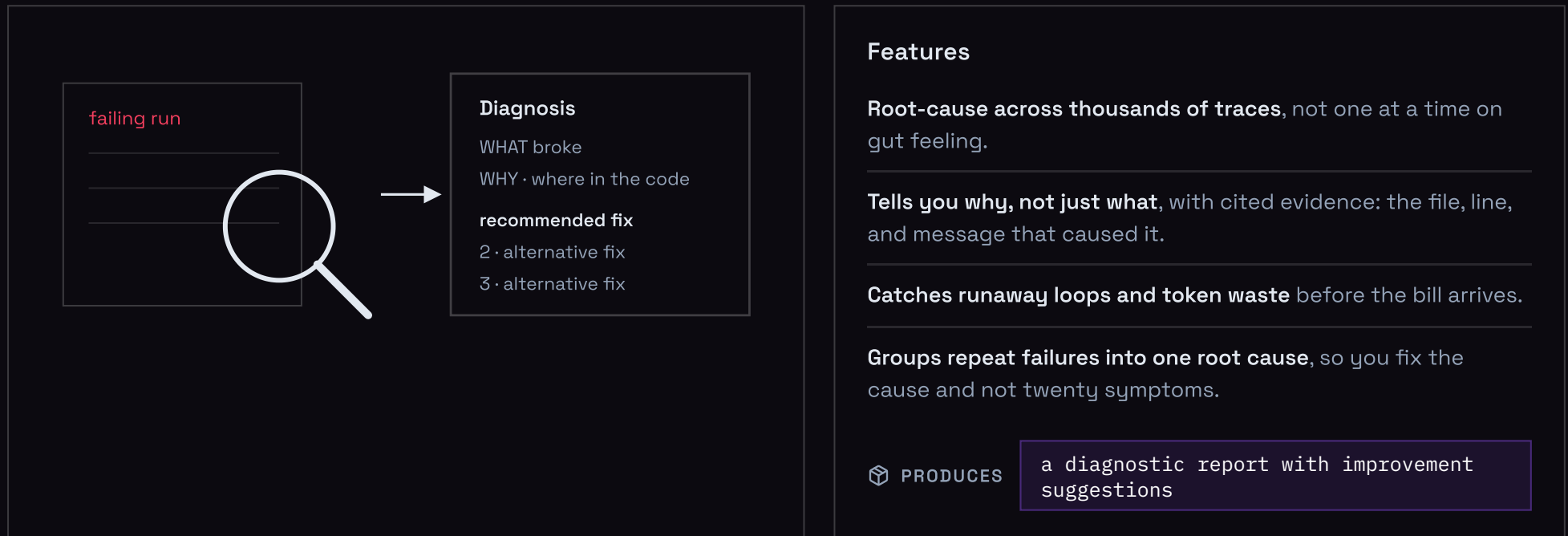
📦 PRODUCES

a calibrated eval suite and evaluation reports from the judge

AGENT
evaluator
judge + criteria

④ Diagnose: when an eval fails, find out why

> ***diagnose** "diagnose the Support Triage agent, refund threads keep failing the policy check"

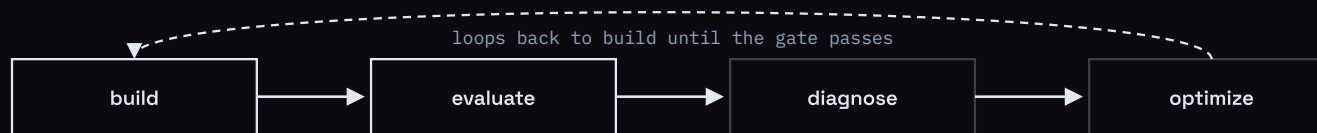


AGENT
diagnostics-analyzer
root-cause analysis

⑤ Optimize: apply the fix, then re-check

› ***optimize** "optimize the Support Triage agent until the refund criterion passes"

The **EDD loop** is the lifecycle's tight inner cycle: **build, evaluate, diagnose, optimize, back to build**, running until the eval gate passes.



Loops until it passes, gated by you

Hands the recommended fix to an AI engineer that edits your agent, then re-runs evaluate. It loops: fix, re-build, re-evaluate, until the agent passes. **Nothing changes without your go-ahead.**

AGENT
ai-engineer
applies the fix

Features

Verify the fix on your real traces before you ship, not after customers complain.

Every fix becomes a permanent regression test, so it can't silently come back.

Ships the way your target expects: a pull request for code, a platform API call for a hosted agent, or a direct update to the agent definition. Nothing changes without your go-ahead.

📦 PRODUCES the updated agent + a fresh scorecard